FlexiGPT: Pruning and Extending Large Language Models with Low-Rank Weight Sharing

James Seale Smith[1] **Chi-Heng Lin**[1] **Shikhar Tuli**[1] **Haris Jeelani**[1]
Shangqian Gao[2] **Yilin Shen**[1] **Hongxia Jin**[1] **Yen-Chang Hsu**[1]
[1]Samsung Research America, [2]Florida State University

arXiv:2501.14713v2 [cs.CL] 31 Jan 2025

Abstract

The rapid proliferation of large language models (LLMs) in natural language processing (NLP) has created a critical need for techniques that enable efficient deployment on memory-constrained devices without compromising performance. We present a method to prune LLMs that selectively prunes model blocks based on an importance score and replaces them with a low-parameter replacement strategy. Specifically, we propose a principled metric to replace each pruned block using a weight-sharing mechanism that leverages unpruned counterparts from the model and block-specific low-rank adapters. Furthermore, we facilitate the learning of these replacement blocks with output feature normalization and an adapter initialization scheme built on low-rank SVD reconstructions. Empirical evaluations demonstrate substantial performance gains over existing methods, achieving state-of-the-art performance on 5/6 benchmarks for a compression rate of 30% and 6/6 benchmarks for a compression rate of 40%. We also demonstrate that our approach can extend smaller models, boosting performance on 6/6 benchmarks using only ≈0.3% tokens of extended training with minimal additional parameter costs.

1 Introduction

The widespread adoption of LLMs has revolutionized NLP applications, driving significant advancements in areas such as virtual assistants, automated customer support, and real-time language translation (Minaee et al., 2024; Naveed et al., 2023). However, deploying these models on memory-constrained devices, such as smartphones and edge devices, remains a formidable challenge due to their substantial parameter sizes and computational demands (Hadi et al., 2023; Raiaan et al., 2024). This paper addresses this challenge by presenting a novel approach that targets *parameter efficiency* to make LLMs more suitable for on-device applications with minimal performance compromises.

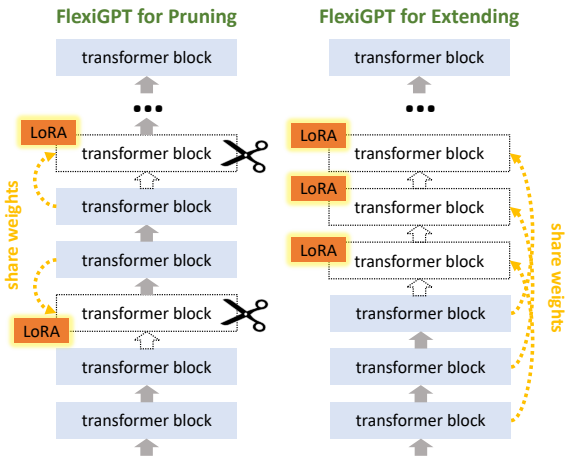

Figure 1: FlexiGPT is used for two settings: (1) pruning a model to reduce parameters with minimal performance cost *or* (2) extending a model to increase performance with minimal parameter cost. **Left:** For pruning models (setting 1), we prune entire blocks and replace them using weight sharing and learned adapters. **Right:** For extending models (setting 2), we repeat block patterns in the model using weight sharing and learned adapters.

Parameter efficiency is particularly critical as it directly impacts the feasibility of deploying LLMs on devices with limited memory and storage resources. Recent model pruning techniques, such as SliceGPT (Ashkboos et al., 2024), LLM Surgeon (van der Ouderaa et al., 2024), LLM-Pruner (Ma et al., 2023), LaCo (Yang et al., 2024), and ShortGPT (Men et al., 2024), reduce the number of parameters but often result in significant performance degradation with minimal recovery after pruning. This gap in existing techniques underscores the need for an end-to-end pruning method that not only reduces the model size but also facilitates performance recovery. *In this work, we propose to recover performance by utilizing existing weights within the model.*

Specifically, we introduce a comprehensive pruning strategy combined with an innovative

weight sharing technique and Low-Rank Adapters (LoRA) (Hu et al., 2021), facilitating efficient parameter usage while preserving the model's performance. We begin by pruning model blocks based on ShortGPT's Block Influence (BI) score (Men et al., 2024). To replace the pruned blocks, we introduce a low-parameter weight-sharing mechanism that leverages existing block modules within the model and incorporates block-specific LoRA parameters, ensuring the selected replacement blocks have high similarity to the pruned blocks while maintaining block diversity. Furthermore, we introduce a novel method to initialize the LoRA adapters in weight-sharing blocks, setting them to be the low-rank difference between the pruned block and the weight-shared replacement block. This initialization minimizes initial disruptions and facilitates smoother model adaptation. Finally, we incorporate *output* feature normalization for pruned blocks to ensure a smooth transition and adaptation, allowing the model to gradually learn and stabilize its performance over time.

Empirical evaluations of our method, which we refer to as *FlexiGPT*, demonstrate substantial performance gains over existing methods. Specifically, we achieve state-of-the-art performance on 5/6 benchmarks for a compression rate of 30% and 6/6 benchmarks for a compression rate of 40% for the popular LLaMA-2 7B model (Touvron et al., 2023). As visualized in Figure 1, our proposed technique not only effectively prunes large models for on-device deployment but *also extends smaller models*, improving their performance at minimal additional parameter costs. Specifically, our method shows that a 22-layer TinyLLaMA (Zhang et al., 2024) model can be extended with repeated blocks, boosting performance on 6/6 benchmarks using only ≈0.3% tokens of extended training with minimal additional parameter costs. *In summary, we make the following contributions*:

1. We develop a weight-sharing technique using adapters and low-rank SVD reconstructions to replace pruned blocks effectively.
2. We apply output normalization to maintain stability and enable gradual learning post-pruning.
3. We propose a method for extending smaller models by repeating layers with unique adapters and normalization parameters.
4. We achieve significant empirical performance gains, achieving state-of-the-art performance on several benchmarks for a variety of models.

2 Background and Related Work

Pruning - Pruning selectively removes less important parameters, reducing model size and computational complexity while maintaining performance. Its greatest benefit lies in optimizing LLMs for deployment in resource-constrained environments such as mobile devices, facilitating faster inference. Several works have been proposed to reduce the size of LLMs by pruning model structures. LLM-Pruner (Ma et al., 2023) removes unimportant coupled structures and the importance is calculated from Taylor expansions. SliceGPT (Ashkboos et al., 2024) applies orthogonal projections to the feature maps and then it performs pruning in the projected space. LLM Surgeon (van der Ouderaa et al., 2024) periodically updates model weights and structures, resulting in a higher cost compared to other methods. Besides reducing the width of the model, ShortGPT (Men et al., 2024) is proposed to remove blocks by using Block Influence scores, and LaCo (Yang et al., 2024) is proposed to collapse layers. Existing pruning methods primarily focus on removing redundant model weights, often neglecting the loss of model capacity. Our approach addresses this limitation by sharing weights from the pruned model to restore its capacity.

PEFT - Parameter-Efficient Fine-Tuning (PEFT) methods aim to mitigate the extensive computational and memory demands of fine-tuning large models by focusing on a smaller subset of parameters. One prominent category of PEFT methods is *adapters*, which involves adding trainable modules to the existing frozen layers of the model (He et al., 2021; Houlsby et al., 2019). Another significant category is *prompt* methods, which augment the initial input sequence with additional trainable vectors known as prompts. This technique focuses on fine-tuning these added tokens rather than the entire model, as demonstrated in works such as Lester et al. (2021); Liu et al. (2021).

Recently, LoRA (Hu et al., 2021) has emerged as the most efficient and highest-performing PEFT approach. LoRA introduces the use of low-rank matrices to adjust model weights efficiently, merging with pre-trained weights before inference to maintain the model's operational speed. Building on this, DoRA (Liu et al., 2024a) decomposes pre-trained weights into magnitude and direction components for fine-tuning, focusing on fine-tuning the directional components using LoRA. *Our research extends these low-rank PEFT methods* by incorpo-

rating LoRA and normalization for efficient weight sharing. Similar to DoRA, our method involves a normalization stage; however, we normalize in the *feature-space* instead of the *weight-space*.

Weight-Sharing - For GPT LLMs, Dehghani et al. (2018) proposed to share all the layers with a dynamic halting mechanism to improve accuracy on the downstream tasks. However, it requires the number of parameters of the base layer (unshared) to match the number of parameters of all layers of vanilla transformers (Vaswani et al., 2017). Subformer (Reid et al., 2021) applies a sandwich-like method of parameter sharing where only middle layers are shared but it does not use any adapter. (Takase and Kiyono, 2021) developed efficient cyclic sharing patterns to increase the accuracy, however their sharing patterns are mainly based on ablation studies. MobileLLM (Liu et al., 2024b) proposed sub-billion parameter architectures for mobile devices and adopted immediate block-wise weight sharing for further accuracy improvement. However, they do not use any pruning and adapters. Cao et al. (2024) introduced matching functions to develop head-wise shareable attention in a principled fashion. Although they use pretrained weights for faster convergence, their matching functions are only applied to share weights among multiple heads of the same layer.

SVD - The Singular Value Decomposition (SVD) of a matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$ is, $\mathbf{W} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, where $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices, and $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is a diagonal matrix of singular values. SVD is widely used to obtain a low-rank representation of $\mathbf{W}$ by selecting the k most significant singular values and their corresponding singular vectors, where $k < \min(m, n)$. Hence, the low-rank representation of $\mathbf{W}$ is given as: $\mathbf{W}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T$, where $\mathbf{U}_k \in \mathbb{R}^{m \times k}$, $\mathbf{\Sigma}_k \in \mathbb{R}^{k \times k}$, and $\mathbf{V}_k \in \mathbb{R}^{n \times k}$, have lower dimensions than $\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}$, respectively. SVD has been applied for model compression (Denton et al., 2014; Hsu et al., 2022) and is closely related to LoRA methods for reducing fine-tuning overhead.

3 FlexiGPT

In this section, we detail our approach, FlexiGPT (Figure 3), which prunes and extends LLMs using LoRA adapters, weight sharing techniques, and output feature normalization. Our method focuses on achieving parameter efficiency while minimizing performance degredation, particularly for memory-constrained devices.

Our method is based on the transformer architecture (Vaswani et al., 2017), which consists of Multi-Head Self-Attention (MHSA) and Multi-Layer Perceptron (MLP) layers. However, our approach is not constrained to this architecture. In general, we refer to blocks (MHSA+MLP), layers (MHSA or MLP), and weights (denoted as W), as our method affects the weights in a uniform manner.

3.1 Pruning Strategy

Our pruning strategy aims to identify and remove blocks that minimally impact the model's performance. To achieve this, we leverage the ShortGPT (Men et al., 2024) Block Influence (BI) score, which has been shown to effectively measure the importance of each block. The Block Influence (BI) score (Men et al., 2024) BI_i for a block i is defined as follows:

$$\text{BI}_i = 1 - \mathbb{E}_{X,t} \frac{X_{i,t}^T X_{i+1,t}}{||X_{i,t}||_2 ||X_{i+1,t}||_2}, \qquad (1)$$

where $X_{i,t}$ denotes the tth row of X_i, and X_i represents the hidden states matrix at block i, with dimensions $T \times d$, where T is the sequence length and d is the hidden dimension. This score captures the extent to which each block transforms its input, with higher scores indicating more significant changes. We calculate the BI score for each block in our model using the validation MiniPile (Kaddour, 2023) subset of the Pile dataset (Gao et al., 2020), and prune the blocks with the lowest BI scores.

We tried other criteria to select blocks for pruning that considered a block's replaceability by another block in the model. However, we found that the BI score results in higher performance on downstream tasks. Our intuition is that the BI score prunes blocks deeper along the model's depth in a sequence, leaving much of the model intact and in the same order, which may explain how it retains strong downstream performance.

3.2 Selection of Weight Sharing Bases

To replace pruned blocks, we aim to find similar unpruned blocks in the model which, when paired with adapters, can recover much of the performance lost after pruning. We aim to select each pruned block's weight sharing 'base' by identifying *similar* unpruned weights. However, a naïve approach such